

## Senior web developer (.NET) tech test

### Welcome

---

Thanks for getting this far. This stage is a build, not an essay. We want to see how you design, write and defend real .NET code, the way you would on a live client project.

Spend no more than four hours on it. A focused partial solution with clear reasoning beats a rushed complete one. If you run out of time, stop and tell us in your README what you would have done next.

Treat the scenario below as an engagement you have just been handed.

### The scenario

---

#### The client

Our client, who we will call Kelvinvale wealth, is a UK wealth manager. It is regulated by the FCA, and it manages investments on behalf of its customers through a network of advisers.

Kelvinvale offers three products: a stocks and shares ISA, a general investment account (GIA) and a self-invested personal pension (SIPP).

#### What they are building

Kelvinvale is replacing a process that runs on phone calls, spreadsheets and paperwork with a portal. One platform, two audiences, and each audience needs a different set of rights over the same records.

- **Customers** log in, keep their personal details up to date, view the products they hold and place instructions against them. They cannot open a new product.
- **Advisers** view existing customers, add new customers and open products for the customers they look after.

# EQUATOR

## What the service has to do

You are building the API behind both journeys. The two audiences reach overlapping resources with different rights over them, so the authorisation model sits at the heart of this task.

## Your task

---

Build a .NET Web API that meets the six requirements below. Use whatever recent .NET version you are comfortable with, and structure the solution however you would structure it for a client.

1. **Model the two audiences.** One API serves advisers and customers. Decide how the caller's identity and role reach your domain, and where you enforce them.
2. **Enforce ownership, not just role.** An adviser reaches only the customers they look after. A customer reaches only their own record and their own holdings. No customer should ever read or move another customer's money.
3. **Open products as an adviser.** Advisers create customers and open ISA, GIA and SIPP products against them, under the rules below. Model those rules so that adding a fourth product type does not mean rewriting the pipeline.
4. **Let customers manage what they own.** Customers update their personal details, view their products and holdings, and place an instruction against a product they already hold.
5. **Prove it with tests.** We care most about the authorisation negative paths and the product rules. A test proving a customer cannot reach someone else's ISA is worth more to us than one proving a happy-path request returns 200.
6. **Make it supportable.** Kelvinvale's compliance team will ask who changed what, on whose account, and when. Give them an answer. Add structured logging and a health endpoint as well.

## The domain rules

---

These are Kelvinvale 's rules:

# EQUATOR

## The three products

- **ISA.** A customer may hold one ISA per tax year. Subscriptions across a tax year cannot exceed the annual allowance.
- **GIA.** No allowance and no eligibility test. Gains are taxable, so every instruction has to be reportable afterwards.
- **SIPP.** The customer must be 18 or over to hold one, and no withdrawal is allowed before the minimum pension age.

## Calls we have deliberately left to you

- How to handle an instruction that would take an ISA past the annual allowance.
- Whether the allowance is checked when the instruction arrives or when it settles.
- What happens to an in-flight instruction that crosses the tax year boundary.
- What a caller sees when a rule refuses their instruction.

Make the decision in the code, explain it in the README, and be ready to chat through with us.

## The caller

Stub the caller rather than integrating an identity provider. Headers are fine, and so is a fake authentication handler. In production these values would arrive as claims.

```
GET /api/v1/customers/{customerId}/products
X-Caller-Id: 3f9c1b4e-7d21-4a55-9b02-1c6e8ad47f10
X-Caller-Role: Adviser
```

## Instructions

An instruction is a customer's request to move money within a product they hold. Type is one of Subscription, Withdrawal or Switch.

# EQUATOR

```
POST /api/v1/products/{productId}/instructions

{
  "type": "Subscription",
  "amountPence": 500000,
  "fundCode": "GLB-EQ-ACC",
  "clientReference": "a7f3-2026-04-02-0001"
}
```

In your solution, seed a handful of advisers, customers and products.

## What to write up

---

Add a README of no more than a page, covering four things.

- Your authorisation model, the decisions behind it and the trade-offs you accepted.
- What you would tackle next given another day.
- How you would host and deploy this on Azure, and what your pipeline would run on every pull request.
- How you used AI, as set out below.

## Using AI

---

AI-assisted development is core to how we deliver, so use the tools you normally use. We are not testing whether you can code unaided. We are testing whether you own what ships.

Name the tools in your README and say where you leaned on them. Be ready to discuss anything you rejected or corrected, because that is usually the most interesting part of the story. Expect to explain any line of the codebase when we talk.

## What we are looking for

---

- Your authorisation model holds up when we push on it from the wrong account.
- Your tests would catch a real regression, including the ones compliance would care about.

# EQUATOR

- A colleague could pick the code up on Monday: clear naming, sensible structure, readable commits.
- You are honest about gaps. Tell us what you skipped rather than hiding it.

We are not scoring you on a user interface, a real identity integration, payment or custody integrations, containers or exhaustive coverage. Persistence is your call, and in-memory is fine. Skip anything the brief does not ask for and spend the time where it counts.

*You will not have every detail, and that is deliberate. Where something is unclear, state the assumption in your README and move on. We want to see how you reason under uncertainty.*

## What happens next

---

Send us a git repository, either a link or a zip, that runs from a clean checkout with one documented command.

Get it to [carole.loga@eqtr.com](mailto:carole.loga@eqtr.com) within 24 hours of receiving the challenge.

You will then meet up to three of the team. We will walk through your code together, talk about the decisions behind it.

Good luck, and thank you for the time you are putting in. We are looking forward to seeing what you build.